矩陣理論及應用　期末報告

JPEG 壓縮對影像奇異值與低秩結構之影響分析

作者　林易辰

學號　B143012007

學校／系級　〔待填〕

授課教師　〔待填〕

繳交日期　〔待填〕

摘要

同一張影像經過較強的 JPEG 壓縮，可能離原圖更遠，卻更容易由少數秩一項近似。本研究以這個表面矛盾為起點，分別量測像素保真度、解碼圖自身的低秩誤差，以及沿原圖秩一方向的係數變化。我們將一張灰階影像獨立以四組品質因子編解碼，對原圖與各解碼圖做全域奇異值分解。截斷重建的平方 Frobenius 殘差與尾端奇異值平方和相符。四組設定的 PSNR 隨品質因子提高而上升，但解碼圖自身的 Rank-100 相對平方誤差並未單調變化。固定原圖方向後，部分係數可為負，補充了奇異值曲線無法表達的方向差異。這些結果限於所用影像及編碼設定。

關鍵字　奇異值分解、秩一矩陣、低秩近似、Frobenius 範數、JPEG

1　從影像壓縮提出矩陣問題

看兩張外觀接近的影像時，我們容易把「看起來像」和「可以用更簡單的矩陣表示」混在一起。工程上這兩件事有不同用途。若目標是傳輸後忠實還原，要關心的是解碼圖離原圖有多遠。若目標是用少量成分表示解碼圖，則要關心它自己的矩陣結構。即使兩張圖的低秩近似誤差接近，它們也可能保有不同的細節或方向。JPEG 提供一個可以在固定輸入下改變編碼設定、再檢驗這些問題的案例。

本文依序回答三個問題。第一，影像為何能拆成一組秩一矩陣，保留前幾項的誤差如何驗證？第二，JPEG 解碼圖相對原圖的像素失真，和它相對自身的低秩誤差，會不會隨品質設定一起變動？第三，若始終用原圖的方向作為尺規，解碼後有哪些成分的係數變了？研究只使用一張灰階圖，範圍是影像矩陣與 JPEG 的關係。

2　從秩一外積到最佳低秩近似

2.1　把一幅圖拆成可以看懂的成分

令 A 為原始灰階影像矩陣，A_{rs} 是第 r 列、第 s 欄的亮度。先想像一個最簡單的影像成分：取一個直向向量 u 和一個橫向向量 v，外積 uv^{T} 的元素是 $u_r v_s$。這個矩陣每一列都沿用同一條橫向輪廓，只改變倍數。以 $u = (1, 2)^{\mathsf{T}}$、$v = (1, 3)^{\mathsf{T}}$ 為教學例，第二列是第一列的兩倍，故矩陣只有一個獨立方向。

$$uv^{\mathsf{T}} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 1 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ 2 & 6 \end{bmatrix}. \tag{1}$$

這個例子說明秩一項為何常先呈現沿列與欄的粗略變化。真實影像的斜邊或交錯紋理通常需要多個項相加，才能逐步補足。

對一般 $m \times n$ 實矩陣 A，令 $p = \min(m, n)$。精簡 SVD 寫作

$$A = U\Sigma V^{\mathsf{T}} = \sum_{i=1}^{p} \sigma_i D_i, \tag{2}$$

$$D_i = u_i v_i^{\mathsf{T}}, \qquad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0. \tag{3}$$

U 的欄 u_i 是左奇異向量，V 的欄 v_i 是右奇異向量，兩組各自正交歸一。Σ 為對角矩陣，對角值 σ_i 是各外積項的非負權重。D_i 的秩為一，$\sigma_i D_i$ 在權重為正時仍為秩一。實作使用 NumPy 的精簡分解 [3]。

2.2 截斷後，誤差落在哪裡？

若只能保留 k 項，其中 $0 \leq k \leq p$，我們以最大的權重開始重建。k 代表保留項數，重建矩陣的秩至多為 k。為了量化被捨棄的像素變化，使用 Frobenius 範數，即所有元素平方和的平方根。對兩個同尺寸矩陣 X, Y，其 Frobenius 內積 $\langle X, Y \rangle_F$ 是逐元素相乘後加總。因此

$$A_k = \sum_{i=1}^{k} \sigma_i D_i, \tag{4}$$

$$\langle D_i, D_j \rangle_F = (u_i^{\mathsf{T}} u_j)(v_i^{\mathsf{T}} v_j) = \delta_{ij}. \tag{5}$$

δ_{ij} 在 $i = j$ 時為 1，否則為 0。這表示每個外積方向的 Frobenius 長度為一，方向彼此垂直。從 A 扣掉 A_k 後只剩尾端項，將殘差平方展開時，所有交叉項消失：

$$\|A - A_k\|_F^2 = \sum_{i=k+1}^{p} \sigma_i^2. \tag{6}$$

式 (6) 提供兩條獨立的數值路徑。一條從像素殘差計算，另一條只讀奇異值。後者也說明對同一張圖，每多保留一項，誤差只會維持或下降。

2.3 為何沒有其他秩至多 k 的矩陣做得更好？

式 (6) 先告訴我們截斷的誤差是多少。要得到最佳性，還必須和所有同樣受秩限制的候選比較。以下以任一實矩陣 X 說明，令它的奇異值為 s_i、左奇異向量為 a_i。任取秩至多 k 的候選 Y，用 P 表示投影到 Y 欄空間的正交投影。因為 $PY = Y$，誤差可拆成兩個正交部分：

$$\|X - Y\|_F^2 = \|(I - P)X\|_F^2 + \|PX - Y\|_F^2 \tag{7}$$

$$\geq \|X\|_F^2 - \|PX\|_F^2. \tag{8}$$

第一項是該欄空間無法接住的能量。第二項是在該空間內仍有的差距。即使第二項已降為零，第一項仍限制了能做到的最好結果。

將 X 的 SVD 代入投影能量，右奇異向量的正交性使得 $\|PX\|_F^2 = \sum_i s_i^2 w_i$，其中 $w_i = \|Pa_i\|_2^2$。每個 w_i 介於 0 和 1。把 a_i 補成完整正交基後，全部投影長度平方之和等於投影空間的維度，因此 $\sum_i w_i \le k$。在總權重有限的情況下，優先把權重給最大的 s_i^2，可得 $\|PX\|_F^2 \le \sum_{i \le k} s_i^2$。將此上限代入式 (8)，任何候選的平方殘差至少是尾端平方和。截斷 SVD 正好達到下界，這就是 Frobenius 範數下的 Eckart–Young 最佳近似結果 [1]。截斷點若切過相等的正奇異值，最優近似可以不唯一。

3 實驗如何把三個問題分開

JPEG 編碼與全影像 SVD 都涉及變換，卻使用不同的基底及範圍。JPEG 的 DCT 模式在局部 8×8 區塊使用固定的餘弦基底，量化步長讓部分係數在除法取整後改變。解碼再以反量化與反轉換回到像素 [2]。SVD 則依整幅影像求出自己的奇異向量。我們用 SVD 看 JPEG 解碼後的全域結構。區塊的高頻位置與全域奇異值的尾端索引沒有一一對應關係。

表 1: 本研究中 JPEG 與 SVD 的工作分工

比較	JPEG 的 DCT 與量化	全影像 SVD
作用範圍	局部 8×8 區塊	整張 512×512 矩陣
方向來源	固定餘弦基底	由各影像的像素決定
實驗角色	編碼、實際解碼與量測檔案大小	分析奇異值、低秩近似與方向係數

原圖是 SciPy 的 8 位元灰階 ascent [5]。每次從同一份原圖像素獨立編碼，品質因子 Q 取 40、60、80、95，並關閉 progressive 與 optimize [4]。將實際 JPEG 檔重新開啟後，記解碼矩陣為 B_Q。此處的 Q 是編碼器設定，儲存成本以檔案的實測位元組數表示。原圖及各解碼矩陣均以 float64 分析，保留秩取 1、5、10、20、50、100。

圖 1 說明哪些量共用原圖，哪些量由每張解碼圖各自取得。像素品質需要把 B_Q 與 A 配對。自身低秩誤差只分解 B_Q。固定方向投影則先在 A 上選定方向，再把同一組方向用於每個 B_Q。這個分流也決定後續各表的解讀基準。

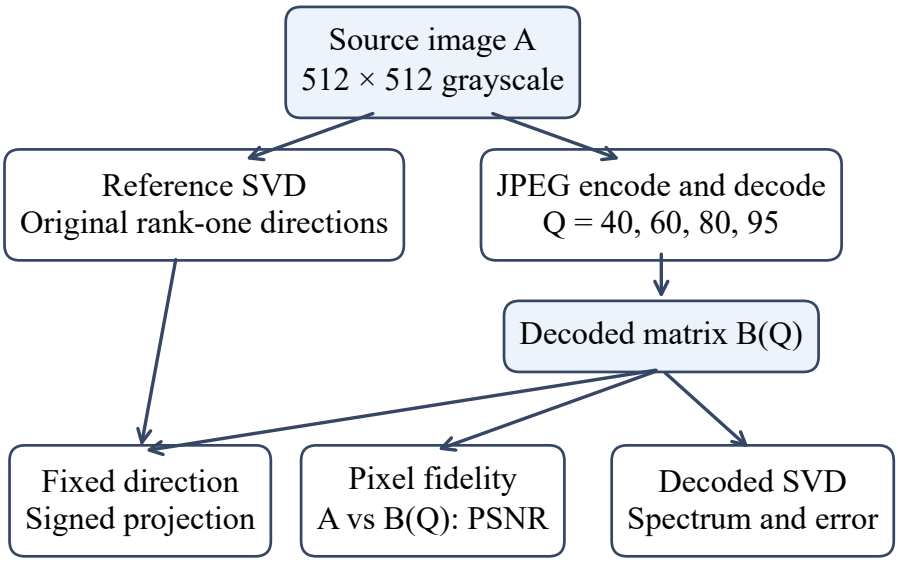


圖 1: 研究資料流。上方原圖分成參照 SVD 與獨立 JPEG 編解碼兩條路徑。下方依研究問題選擇對應的矩陣與方向。

4 結果：像素保真度與低秩誤差

4.1 先問解碼圖離原圖多遠

用逐像素誤差計算 MSE，再將它相對 8 位元峰值 255 轉換成 dB。對 $m \times n$ 的原圖與解碼圖：

$$\mathrm{MSE}(A, B_Q) = \frac{\|A - B_Q\|_F^2}{mn}, \tag{9}$$

$$\mathrm{PSNR}(A, B_Q) = 10 \log_{10}\left(\frac{255^2}{\mathrm{MSE}(A, B_Q)}\right). \tag{10}$$

MSE 為零時 PSNR 為無限大。PSNR 越高代表這種平方誤差度量下，解碼像素越接近原圖。它沒有直接描述輪廓辨識、語意或觀看者感受。

如果提高品質設定，增加的檔案大小換得多少像素保真度？圖 2 以實際檔案大小為橫軸、PSNR 為縱軸，保留四個實測點。從 $Q = 40$ 到 $Q = 95$，檔案與 PSNR 均增加，兩者的精確值見表 2。較細的量化通常讓反轉換後的像素誤差較小，與本案例的觀察相容。四個點只能描述這張圖的結果，無法據此推定其他影像的儲存成本或 PSNR。

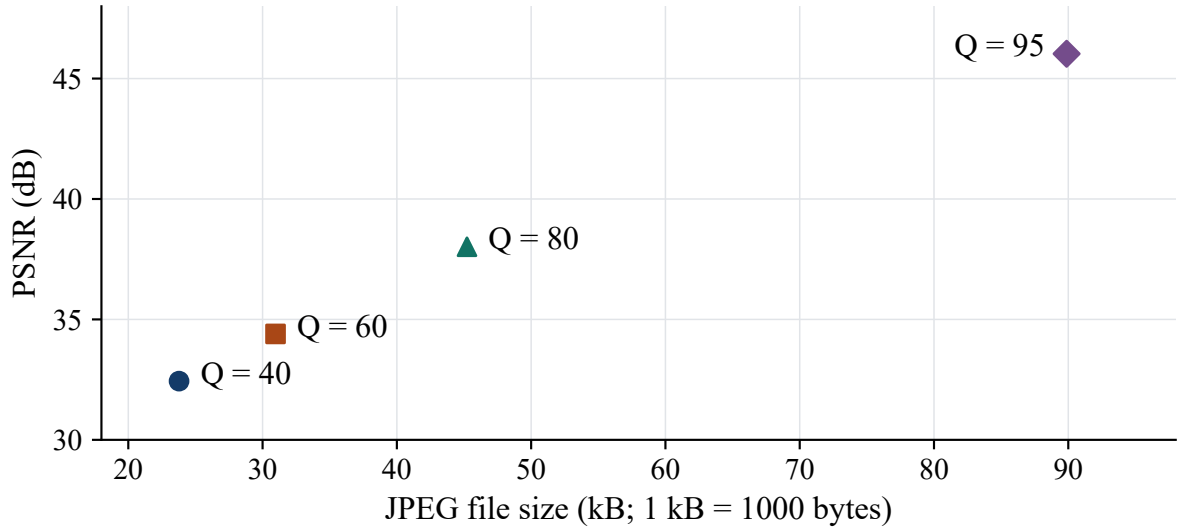


圖 2: 實測 JPEG 檔案大小與相對原圖的 PSNR。不同符號代表不同品質因子，橫軸以每 1000 位元組為 1 kB。

4.2 再問每張解碼圖本身是否容易近似

對非零矩陣 X，令 $s_i(X)$ 為其奇異值。前 k 項的累積平方能量占比 $E_k(X)$，以及最佳截斷的自身相對平方誤差 $R_k(X)$ 定義如下：

$$E_k(X) = \frac{\sum_{i \le k} s_i(X)^2}{\sum_i s_i(X)^2}, \tag{11}$$

$$R_k(X) = \frac{\|X - X_k\|_F^2}{\|X\|_F^2} = 1 - E_k(X). \tag{12}$$

分母是這張矩陣自身的總平方能量。零矩陣的絕對重建誤差為零，相對誤差因分母為零而未定義。本研究的五張影像矩陣均非零。$R_k(B_Q)$ 越小，代表 B_Q 越容易用它自己的前 k 項重建。

表 2: 四種 JPEG 品質設定的實測結果。末欄以每張解碼圖自身為基準，原圖同一指標為 0.51677%。資料定位見表 3。

Q	檔案（bytes）	MSE	PSNR（dB）	$R_{100}(B_Q)$
40	23,780	37.09696	32.43742	0.46522%
60	30,965	23.62286	34.39748	0.52460%
80	45,222	10.26691	38.01641	0.53177%
95	89,876	1.62239	46.02924	0.52482%

圖 3 將同一組 Q 上下對齊，上圖回答像素保真度，下圖回答自身低秩近似性。$Q = 40$ 的 PSNR 為 32.43742 dB，是四組最低，R_{100} 卻是四組中最小的 0.46522%。

相對地，$Q = 95$ 的 PSNR 最高，R_{100} 也未沿著上圖的順序變動。下圖虛線代表原圖自身的誤差，只作結構參照。這個對比說明，用自身低秩誤差代替對原圖的失真，會改變研究問題。

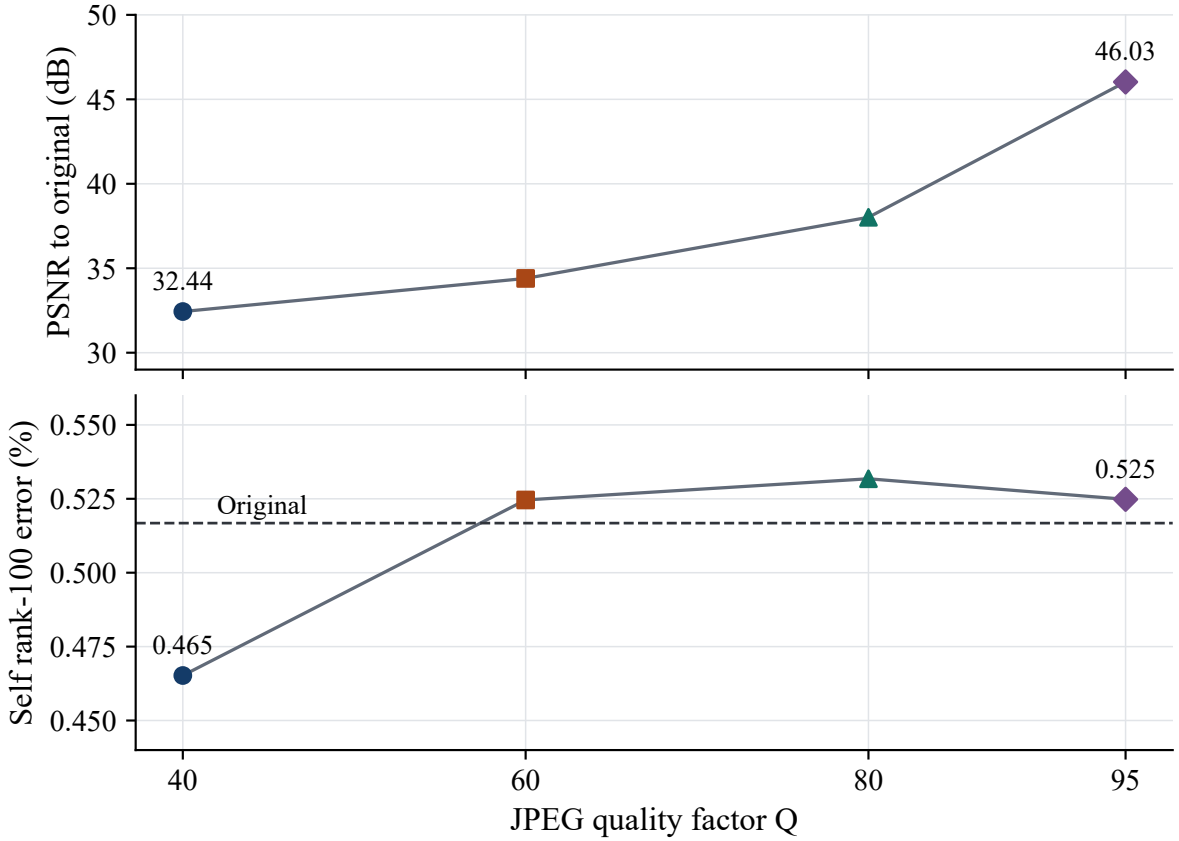


圖 3: 同一品質因子下的兩種比較基準。上圖以原圖為基準，下圖以各解碼圖自身為基準。下圖虛線為原圖自身 Rank-100 誤差。

4.3 近似差異出現在多少個成分之後？

若只比較 $k = 100$，容易忽略五張圖其實有相近的整體下降形狀。圖 4 上方呈現完整的取樣秩，依式 (6)，固定矩陣的誤差隨保留項增加而下降。下方只看 $k = 50, 100$，將解碼圖自身的誤差百分比減去原圖在同一個 k 的誤差百分比。零線代表兩者一樣易於近似，負值代表解碼圖的自身誤差較低。每張圖仍使用各自的總能量作分母，差值接近零無法證明解碼像素也接近原圖。

要理解 R_{100} 為何不同，可以回到奇異值分布。圖 5 的前段大致重合，尾段的差別較明顯。以索引 400 為例，原圖奇異值為 27.10575，$Q = 40$ 為 6.09311，$Q = 95$ 為 28.75731。同一索引可以變大也可以變小。把前 100 項平方加總再除以各自總能量，原圖為 99.48323%，$Q = 40$ 為 99.53478%，與式 (12) 給出的互補關係相符。較粗的 JPEG 量化可能減弱部分局部細節，讓能量更集中，但這些資料尚無法將特定區

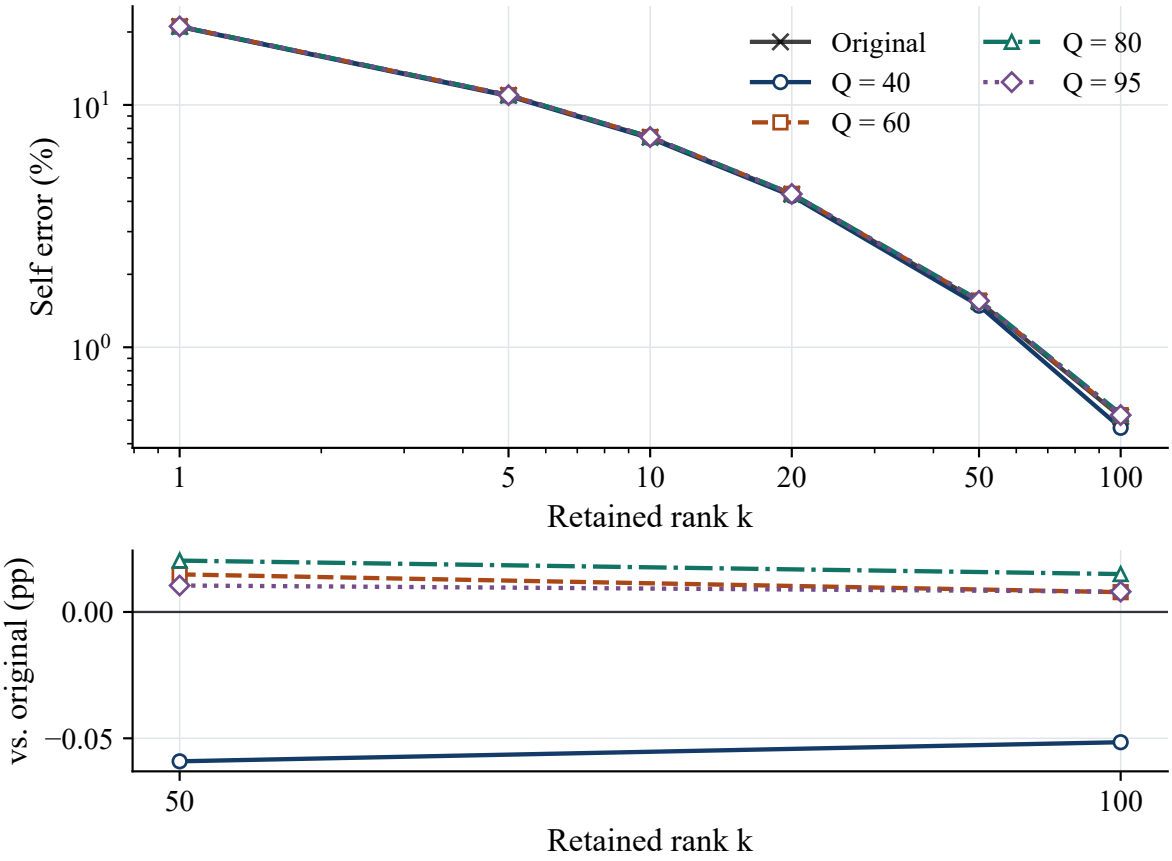


圖 4: 上圖以雙對數尺度呈現自身低秩誤差。下圖以百分點（pp）標示 $100[R_k(B_Q) - R_k(A)]$。線段只連接已計算的秩。

塊頻率的改變歸因到某個全域奇異值。

4.4 從圖像看出截斷留下了什麼

圖 6 將原圖的不同截斷結果放在同一灰階範圍。Rank 1 只留下粗略的橫向與縱向亮暗，增加成分後，斜向樓梯與欄杆逐漸可辨。圖 7 固定裁切窗口後再比較，避免全圖縮小使欄杆差異不易看清。Rank 10 保留部分輪廓，Rank 50 和 Rank 100 逐步補足局部對比。$Q = 80$ 的 JPEG 解碼圖與原圖在相同窗口展示，可用來看像素外觀。這些 PNG 只供觀察，重建誤差仍使用裁切、取整前的浮點矩陣。裁切是既有影像的局部對照，未再做編碼或另計區域指標。

5 固定原圖方向後，係數如何改變？

把 B_Q 重新分解，可以得到它最適合自己的方向。若想知道「原圖的同一個成分，在解碼後變強或變弱多少」，換用解碼圖的方向就失去了共同參照。我們改用式 (2) 的 D_i 作固定方向。它的 Frobenius 長度為一，因而可以用 Frobenius 內積讀取 B_Q 在該

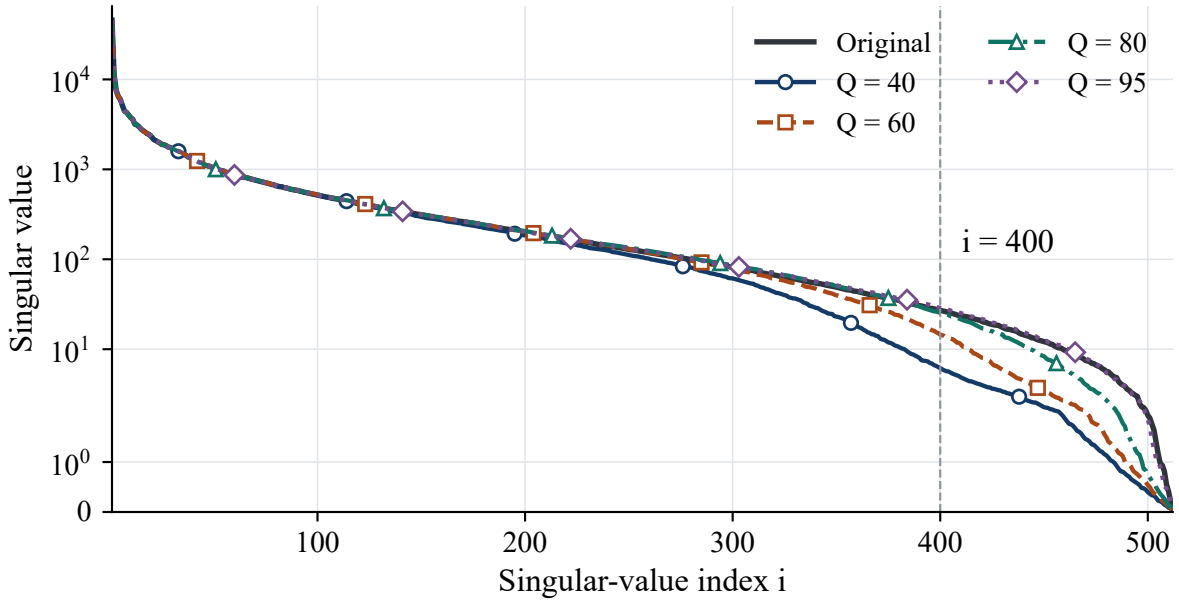


圖 5: 各影像自身的奇異值分布。縱軸使用對稱對數尺度，直線標出文中比較的第 400 項。各線的奇異向量由各影像獨立求得。

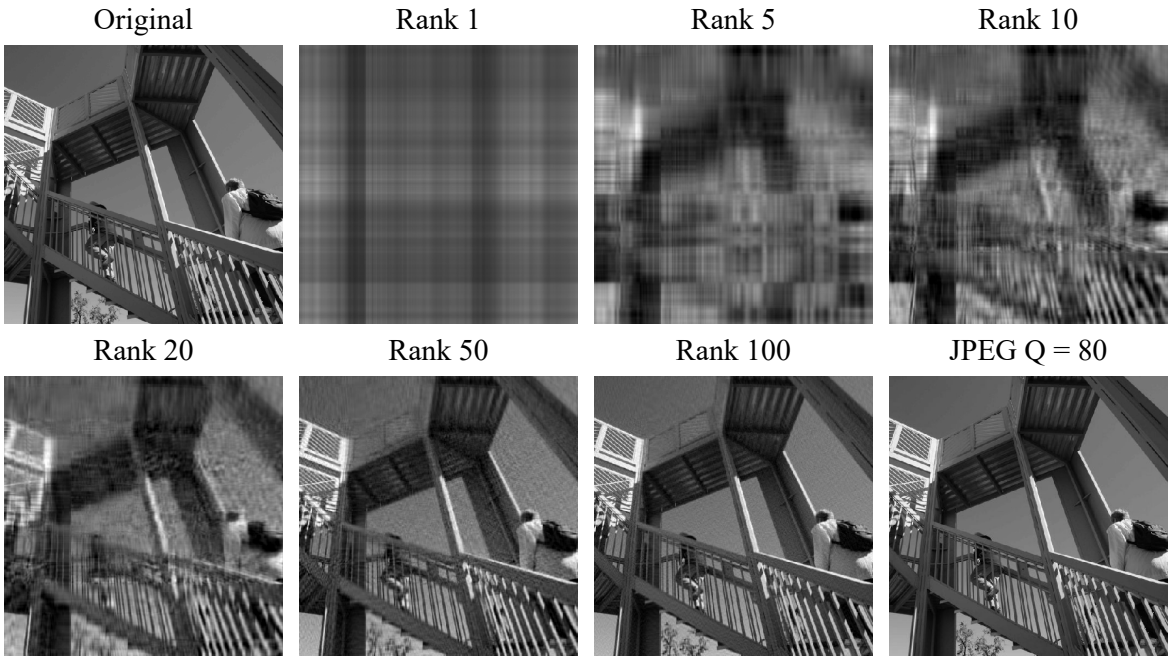


圖 6: 原圖、六組原圖截斷重建與 Q=80 解碼圖。所有面板共享灰階範圍。

方向的有號分量。令 $c_i(Q)$ 為此分量，$\Delta_i(Q)$ 為它相對原圖權重的變化：

$$c_i(Q) = \langle B_Q, D_i \rangle_F = u_i^{\mathsf{T}} B_Q v_i, \tag{13}$$

$$\Delta_i(Q) = c_i(Q) - \sigma_i = \langle B_Q - A, D_i \rangle_F. \tag{14}$$

式 (13) 的第二個等號來自逐元素乘積求和。先沿 v_i 把每列加權，再沿 u_i 合併。若 $B_Q = A$，各個 D_i 的正交性使 $c_i = \sigma_i$。變成其他矩陣後，內積可以是負值，表示沿

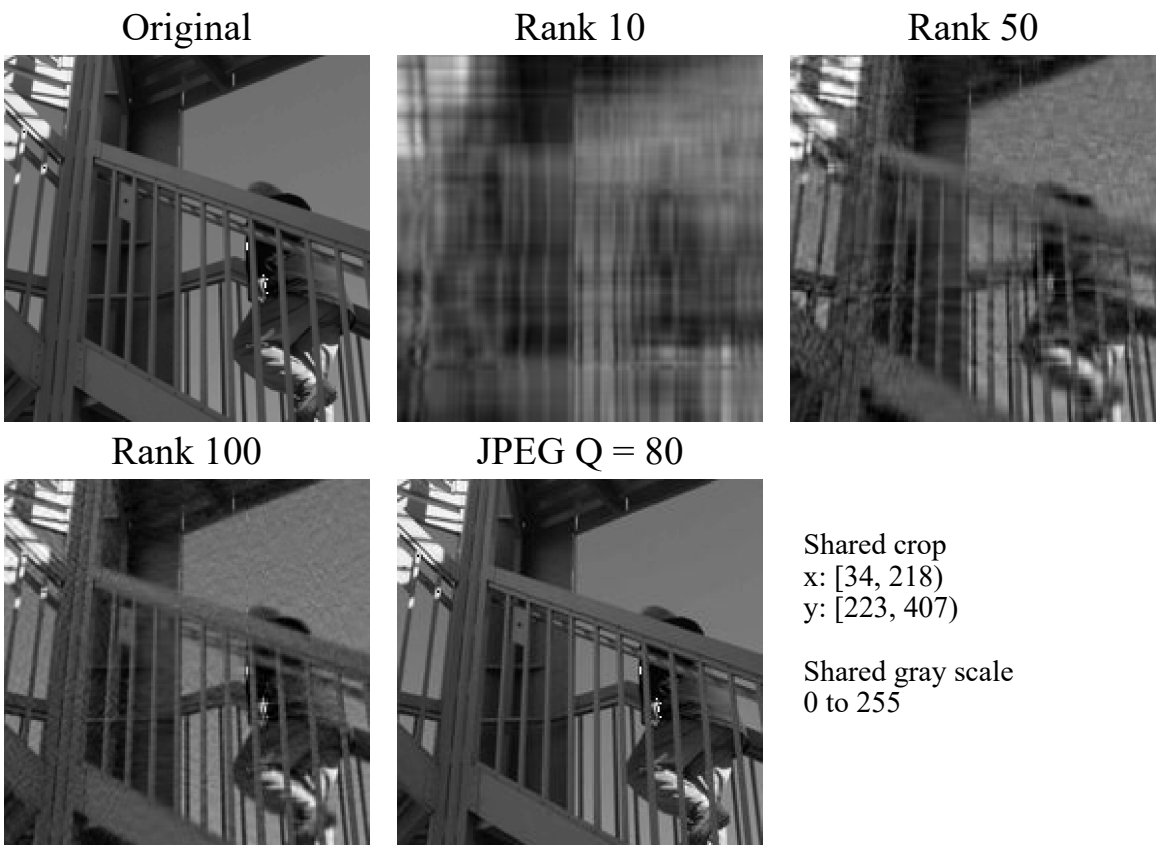


圖 7: 相同位置的局部放大，依序比較原圖、Rank 10、Rank 50、Rank 100 與 Q=80 解碼圖。裁切座標使用零起點，右界及下界不含。

該固定方向的分量符號相反。B_Q 自身的奇異值皆非負，而且重新排序。若原圖有重複奇異值，對應子空間可改選基底，個別係數會依方向而異。這組投影只取原圖基底中配對的 i, i 項，沒有包含所有跨方向項。

圖 8 上方先看差值分布。第 200 項在 $Q = 40$ 為 -75.10047，在 $Q = 95$ 為 -1.43469，說明同一方向可以隨設定有不同幅度的變化。下方改看第 512 項的係數本身。原圖在此方向的奇異值為 0.15024，$Q = 95$ 的係數為 -1.35063。這個負號是式 (13) 的方向資訊，無法由解碼圖自身的奇異值正負看出。個別方向的變化也不足以代替整體像素誤差或資訊保留率。

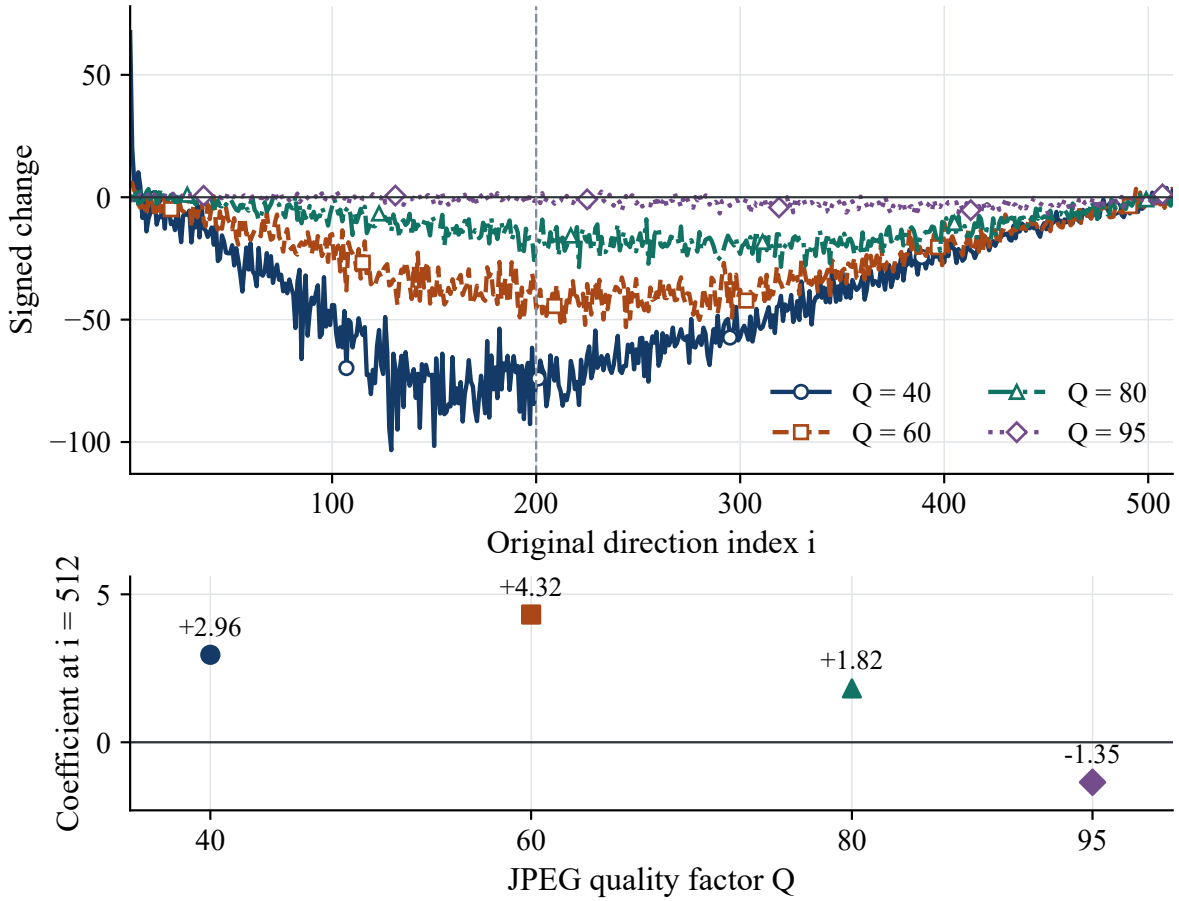


圖 8: 固定原圖方向的變化。上圖為各索引的有號差值，直線標出第 200 項。下圖為第 512 項的有號係數，水平線標示零。

6 結論與限制

第一，原圖與解碼圖都可以從秩一方向的加權和理解。截斷 SVD 的尾端平方和給出直接且可驗證的誤差，也說明為何保留更多成分時能逐步重建更多結構。

第二，對這張圖，提高 JPEG 品質因子改善了相對原圖的像素品質，但自身 Rank-100 誤差未呈單調變化。較強壓縮讓解碼圖更易低秩近似，仍可能使它離原圖更遠。每個指標的分母與比較對象，是解讀差異的關鍵。

第三，固定原圖基底讓我們能追蹤方向上的有號變化。這類投影和各圖自身奇異值互相補充。前者回答同一方向在解碼後如何改變，後者回答解碼圖本身如何分解。兩者都沒有直接量測語意資訊或視覺品質。

研究限於一張灰階圖及指定編碼器。日後可用不同紋理的影像檢查上述現象是否重現。若要延伸到數位浮水印，應先設定嵌入與提取方法，以未壓縮對照確認能取回，再量測 JPEG 後的位元錯誤率與嵌入失真。這些屬於後續研究。

附錄：數值核對與重現

從圖表回到原始資料

表 3 的路徑均相對於專案根目錄，CSV 與實驗清單放在 outputs/run-m2/。報告中的百分比以原比率乘 100，再依顯示位數四捨五入。

表 3: 報告引用與封存資料對照

報告項目	檔案	條件及欄位
表 2、圖 2、圖 3 上	jpeg_metrics.csv	各 Q 的 jpeg_bytes、mse、psnr_db
表 2 末欄、圖 3 下、圖 4	rank_errors.csv	各影像與取樣秩的 relative_squared。末段比較為同秩的解碼圖減原圖。
圖 5、平方能量例子	spectrum.csv	各影像的 sigma（例：$i = 400$）與 cumulative_energy（$i = 100$）
圖 8 與投影例子	projections.csv	各 Q 的 delta（例：$i = 200$）、coefficient（$i = 512$）及同索引的 original_sigma
圖 6 與 7	原圖與已存 PNG	data/ascent.png、六張原圖截斷重建、Q=80 解碼 PNG

用兩條計算路徑驗證同一誤差

存檔中的像素路徑先對各矩陣計算截斷重建，再從像素殘差求 Frobenius 平方誤差。頻譜路徑則直接對尾端奇異值做平方和。兩路徑除以同一矩陣的平方 Frobenius 範數後，依相對容差 10^{-8} 與絕對容差 10^{-12} 比較。M2 五個矩陣的最大差為約 1.11×10^{-15}，見清單 manifest.json 中各矩陣的 max_relative_identity_gap。這項檢查針對式 (6) 的計算一致性。研究詮釋仍須分辨指標的基準。

環境與重跑

M2 存檔使用 Windows、Python 3.13.14、NumPy 2.4.6、Pillow 12.3.0、libjpeg-turbo 3.1.4.1。環境與編碼參數記在 manifest.json。從專案根目錄執行下列 PowerShell 命令。使用尚未存在的輸出目錄。

```
py -3.13 -m venv .venv-final
$python = '.\.venv-final\Scripts\python'
& $python -m pip install -r requirements.txt
& $python -m pip check
Get-FileHash data/ascent.png -Algorithm SHA256
& $python -m experiment --input data/ascent.png `
  --output-dir outputs/run-m4-local --qualities 40 60 80 95
& $python -m pytest -v
```

輸入 PNG 的 SHA-256：

3b7cae537bc09699016556df746184a2376083244ae375a2740ee7157970ebf3

跨平台的 JPEG 函式庫可能改變編解碼結果，BLAS 執行方式也可能影響浮點末位。應以每次實際解碼的像素驗證該次的 MSE、頻譜與投影，並比對清單中的版本與輸入雜湊。圖表重製只讀封存資料，不重新編碼。

參考文獻

1. Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3), 211–218. https://doi.org/10.1007/BF02288367

2. ITU-T. (1992). *Recommendation T.81: Information technology – Digital compression and coding of continuous-tone still images – Requirements and guidelines*. https://www.itu.int/rec/T-REC-T.81/en

3. NumPy Developers. *numpy.linalg.svd*. https://numpy.org/doc/stable/reference/generated/numpy.linalg.svd.html

4. Pillow Contributors. *Image file formats: JPEG*. https://pillow.readthedocs.io/en/stable/handbook/image-file-formats.html#jpeg

5. SciPy Developers. *scipy.datasets.ascent*. https://docs.scipy.org/doc/scipy/reference/generated/scipy.datasets.ascent.html